Code of Conduct:

I AKSHAYA SEKAR (1911260204) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AKSHAYA SEKAR

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

SIMATS ENGINEERING

CO2_AT2 REVERSE ENGINEERING TASK

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED: CO2

Assessment Tool Weightage: 30%

1. Question: A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with a structured explanation.

System decomposition: The ATM withdrawal flow can be divided into card/PIN validation, account selection, amount of entry, balance and withdrawal-limit validation, cash dispensing, account debit, receipt of generation, and error handling. Because source code and actual observations are not supplied, the original test design can only be inferred, not confirmed.

**Equivalence classes: ** For withdrawal amount, use valid positive amounts within the permitted per-transaction limit and available balance; invalid classes include zero, negative amounts, values above the limit, amounts greater than available balance, unsupported denominations, non-numeric input, and blank input. Other classes include

valid/invalid card, correct/incorrect PIN, active/blocked account, and available/insufficient ATM cash.

**Boundary Value Analysis: ** For a configured minimum amount, maximum withdrawal limit, and account balance, test values just below, at, and just above each boundary. For example, if the maximum is L, test L−1, L, and L+1 in valid denomination units; use actual configured limits rather than assuming values.

**Reconstructed tests: ** Verify a normal withdrawal; exact minimum and maximum; amount above maximum; amount exceeding balance; invalid denomination; incorrect PIN; blocked card; insufficient ATM cash; network timeout; and duplicate submission. For each, check response message, account balance, cash-dispensing action, transaction record, and receipt. Random or exploratory tests may supplement these cases, but ECP and BVA provide systematic baseline coverage. The exact limits and expected messages must come from requirements or observed system behavior.

2. Question: A software module execution traces and code snippets are given, but no test documentation is available. Reverse engineer with the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.

A control flow graph (CFG) represents program execution as nodes for statements or basic blocks and directed edges for possible transfers of control. Decision nodes have separate true and false edges; loops require entry, body, exit, and back edges. However, the prompt does not include the actual code of snippets or execution traces, so a specific CFG and exact independent paths cannot be enumerated reliably.

**Reverse-engineering process: ** Map each code block to a node, label every decision outcome, mark loop entry and exit, and identify the program entry and exit. Compare observed execution traces with the graph to determine which nodes and edges were visited. Statement coverage indicates whether statements executed; branch coverage checks each decision outcome; path coverage checks feasible paths and may be impractical to exhaust when loops allow many iterations.

**Improved strategy: ** Test both outcomes of each decision, each loop with zero iterations where valid, one iteration, multiple iterations, and boundary/termination cases. Combine feasible independent paths and risk-prioritized paths, and document expected results. Cyclomatic complexity can help estimate the number of linearly independent paths using the completed CFG. Full path coverage should only be claimed when the feasible path set is bounded, and all required paths have been exercised.

3. Question: A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer for the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.

**Components: ** Login input validation, authentication, session management, authorization, transaction entry, balance/limit checks, transaction confirmation, persistence, and error handling. Defect logs can suggest missed input classes or

control-flow branches, but without the actual logs the original techniques cannot be identified with certainty.

**Likely evidence: ** Repeated failures for invalid formats may indicate incomplete equivalence partitioning; defects at limits may indicate missing boundary tests; failures tied to conditional combinations may indicate weak branch/path coverage. These are hypotheses to check against the real defect records, not confirmed findings.

**Reconstructed strategy: Use ECP for valid and invalid usernames, passwords, transaction amounts, account states, and session states. Use BVA around password-length limits, transaction limits, and available balances as specified. Use path testing for success, invalid credentials, locked account, expired session, unauthorized transaction, insufficient funds, limit exceeded timeout, and rollback/error paths. Include security checks such as repeated login attempts and access-control enforcement where requirements apply. Trace every test to a requirement, record actual and expected results, and prioritize high-risk authentication and financial paths.

4. Question: A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer for the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.

**Functional units: ** Item identification, stock lookup, quantity validation, stock update, threshold comparison, alert generation, transaction logging, and error recovery. Actual log entries were not supplied, so specific hidden rules and observed defects cannot be inferred from evidence here.

**Possible equivalence classes: ** Existing/non-existing item; positive, zero, and negative update quantity; valid/invalid quantity format; resulting stock above, at, or below the low-stock threshold; and sufficient/insufficient stock for an issue transaction. The permitted behavior depends on whether the operation is a receipt, issue, adjustment, or return.

**Boundary tests: ** If the alert threshold is T, test resulting stock at T−1, T, and T+1. Also test stock of zero, an update that makes stock exactly zero, an attempted issue greater than current stock, and maximum permitted quantities if defined. Verify the new stock value, alert trigger, transaction log, and error response. Repeated submissions should be checked for duplicate updates if the system supports retries. Use the actual business rules to set up expected results and compare each case with the supplied logs to identify missing coverage.

5. Question: You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluates its impact on system stability.

A CPU/memory profile can identify functions or periods with unusually high resource use, but it cannot by itself prove the exact source-level loop or decision responsible. The executable and actual profile data are not attached in the prompt, so the graph below is conceptual rather than a reconstruction of the real implementation.

**Conceptual CFG: ** Entry → validate request → locate member/book → decision: record found? → process borrowing/return → decision: eligibility and availability valid? → update records → Log Results → Exit. Error branches handle missing

records, ineligible members, unavailable books, and storage failures. A loop may occur when searching records or processing a collection of books/members.

**Testing strategy: ** Create tests for empty, small, typical, and large datasets; found/not-found records; eligible/ineligible members; available/unavailable books; and repeated operations. Exercise zero, one, and many loop iterations where appropriate. Profile CPU, memory, response time, and resource growth for each path; test repeated operations to detect leaks or unbounded accumulation. Compare results against functional expectations and set performance thresholds from requirements. This combines structural path testing with stress testing and helps identify expensive or unstable paths without claiming an exact CFG absent the code.

6. Question: A mobile payment gateway's API response (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer for the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.

The named API states suggest a state-transition model, but the actual JSON responses and transition triggers are not included. Therefore, legal transitions must be confirmed from the API specification rather than assumed.

**Model elements: ** States are Pending, Success, Reversal, and Timeout. Inputs/events may include payment acceptance, confirmation from the payment network, reversal request, response timeout, retry, and late response. Each transition should specify the triggering event, required fields, resulting state, and response code.

**Conceptual transition diagram: ** Initiated → Pending; Pending → Success on valid confirmation; Pending → Reversal when a permitted reversal occurs; Pending →

Timeout when the timeout condition is met. Any transition from Success, Reversal, or Timeout back to another state must be defined by the specification, including treatment of late confirmations and retries. Do not presume transitions are legal without that definition.

**Test suite: ** Test every specified legal transition and every prohibited transition; missing or malformed transaction IDs; duplicate requests; repeated callbacks; delayed and out-of-order responses; timeout followed by late success; reversal after success; and idempotent retry behavior. Verify state, balance/ledger effects, response of JSON, and audit logs. A transition table should record current state, event, expected next state, and expected response.

7. Question: An encrypted communication module provides only "black box" behavior through its input parameters and encrypted outputs. Reverse engineer for validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.

**Components: ** Input parsing, length/type/range validation, key/nonce or configuration validation, encoding conversion, encryption, output formatting, and error handling. With only encrypted outputs, infer validation behavior by comparing outputs and error responses across controlled input variations; encryption output differences alone may not reveal the internal validation rule.

**BVA test design: ** For each documented input length limit N, test N−1, N, and N+1; also test empty input, minimum valid length, maximum permitted length, malformed encodings, unexpected types, missing fields, and values at numeric bounds.

Use safe, non-destructive malformed inputs and verify consistent rejection, no crash, no truncation, and no sensitive data in error messages.

**Security relevance: ** Enforcing length and type limits, using safe parsing, and rejecting invalid encodings can reduce risks associated with memory misuse and injection. BVA alone cannot prove that buffer overflow or injection vulnerabilities are absent. Combine it with fuzz testing, code analysis where source is available, memory-safety checks, and security testing. The exact limits and expected errors must come from the interface specification.

8. Question: A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer for the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.

One hundred percent of statement coverage only shows that each executable statement ran at least once. It does not ensure that each decision outcome or each Boolean condition was independently tested. For multi-condition decisions, tests may execute the calculation statement while missing important combinations.

**Conceptual CFG: ** Entry → read policy inputs → validate inputs → decision: eligibility conditions satisfied? → (true) calculate base premium → decision: surcharge/discount conditions? → apply adjustment → output premium → exit. False edges should lead to the appropriate rejection or alternate calculation. This is a conceptual graph because the actual code is not supplied.

**Improved tests: ** Create test cases for each true and false outcome of every decision and vary each Boolean condition so it takes both values while other conditions are controlled. Include valid/invalid policy data, minimum/maximum age or coverage values if defined, and combinations that trigger each surcharge and discount. Derive independent feasible paths from the actual CFG and measure branch coverage after execution. Complete path coverage is only practical if the feasible path set is finite and bounded; otherwise document the selected feasible paths and stronger criteria such as condition or modified condition/decision coverage where appropriate. Use approved premium rules as the oracle for expected calculations.

9. Question: A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer for the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer to the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.

**Components: ** File discovery, metadata/type validation, chunking or uploading, network transfer, conflict resolution, server acknowledgement, local state update, retry, and recovery. The actual activity logs and defect reports are not included, so specific patterns cannot be attributed to a confirmed original test strategy.

**Possible equivalence classes: Small/typical/large files; supported/unsupported file types; stable/slow/interrupted networks; new/modified/deleted files; and no-conflict/conflicting updates. These classes should be aligned with documented size limits, type rules, and sync semantics.

**BVA and stress strategy: ** For maximum file size M, test M−1, M, and M+1 where applicable, along with empty files and files near chunk boundaries. Test file

names and metadata at their documented length/format limits. Stress with many files, concurrent updates, slow/intermittent connectivity, retries, and service restarts. Verify file hashes or content equality, metadata, conflict outcomes, and recovery after interruption. Idempotent retries, atomic updates, and reconciliation checks should be tested where supported. This improves coverage of size and network boundaries, but prevention claims require passing tests and validation against the system's actual recovery design.

10. Question: You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.

Search behavior can be inferred by comparing query predicates with returned rows, but the actual SQL statements and datasets are not included. Therefore, the exact filtering logic and completeness cannot be confirmed from the prompt alone.

**Components: ** Parameter parsing, validation, filter construction, AND/OR/NOT evaluation, sorting, pagination, database execution, and result formatting. For each filter, identify whether it is optional, its accepted values, and how it combines with other filters.

**Decision table: ** For two binary filters A and B, enumerate all four combinations: A=yes/B=yes; A=yes/B=no; A=no/B=yes; A=no/B=no. Add columns for the expected inclusion/exclusion of a record and the expected result set. If there are more binary filters, the full combination count is 2^n before constraints or rule reduction; use pairwise or risk-based reduction only when exhaustive combinations are impractical

and justify it. Test AND, OR, and NOT semantics with records designed to match each predicate individually and in combination. Include missing, empty, invalid, and boundary-valued parameters, plus sorting and pagination checks. Compare expected

rows with actual query results and verify parameterized queries that are used. The table is complete only when all defined rules and relevant combinations are represented.

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation